⍰ Guía Completa de Métodos para Manipular Arrays (Vectores) en JavaScript

En esta guía encontrarás todos los métodos fundamentales y modernos para manipular arrays (vectores) en JavaScript. Cada método está explicado con ejemplos claros, indicando si modifica el array original, si devuelve valores útiles, y comparaciones cuando existen métodos similares o en desuso.

➕Métodos para Agregar y Quitar Elementos en los Extremos

push()

Agrega elementos al final del array. Devuelve la nueva longitud.

```
let frutas = ['banana', 'naranja'];
frutas.push('sandía'); // ['banana', 'naranja', 'sandía']
```

unshift()

Agrega elementos al principio del array. Devuelve la nueva longitud.

```
frutas.unshift('manzana'); // ['manzana', 'banana', 'naranja', 'sandía']
```

pop()

Quita el último elemento del array y lo devuelve.

```
let ultima = frutas.pop();
console.log(ultima); // 'sandía'
```

shift()

Quita el primer elemento del array y lo devuelve.

```
let primera = frutas.shift();
console.log(primera); // 'manzana'
```

⍰ Métodos para Agregar, Eliminar o Reemplazar en Posiciones Específicas

splice()

Agrega, elimina o reemplaza elementos. Modifica el array original.

```
let frutas = ['manzana', 'banana', 'naranja'];
let eliminado = frutas.splice(1, 1); // elimina 1 elemento desde índice 1
console.log(frutas); // ['manzana', 'naranja']
console.log(eliminado); // ['banana']
```

⍰ Métodos para Copiar o Extraer porciones (sin modificar)

slice()

Extrae una parte del array. No modifica el original.

```
let frutas = ['manzana', 'banana', 'naranja', 'kiwi'];
let seleccion = frutas.slice(1, 3); // ['banana', 'naranja']
```

---

concat()

Une dos arrays y devuelve uno nuevo.

```
let frutas = ['manzana', 'naranja'];
let verduras = ['lechuga', 'tomate'];
let combinado = frutas.concat(verduras);
```

⍰ Métodos de Búsqueda y Verificación

indexOf()

Devuelve el índice del primer elemento que coincide, o -1 si no existe.

```
frutas.indexOf('naranja'); // 2
```

---

includes()

Devuelve true si el array contiene el valor especificado.

```
frutas.includes('naranja'); // true
```

---

find()

Devuelve el primer elemento que cumple una condición.

```
let numeros = [1, 2, 3];
let encontrado = numeros.find(n => n > 1); // 2
```

---

findIndex()

Devuelve el índice del primer elemento que cumple una condición.

```
numeros.findIndex(n => n > 1); // 1
```

⍰ Métodos para Ordenar o Invertir

sort()

Ordena el array alfabéticamente (como string). Puede usar función de comparación.

```
let numeros = [10, 2, 30];
numeros.sort((a, b) => a - b); // [2, 10, 30]
```

---

reverse()

Invierte el orden del array (modifica el original).

```
let frutas = ['manzana', 'banana'];
frutas.reverse(); // ['banana', 'manzana']
```

⍰ Métodos de Recorrido

forEach()

Ejecuta una función por cada elemento. No devuelve nada.

```
let frutas = ['manzana', 'banana'];
frutas.forEach((fruta, index) => {
  console.log(index + ': ' + fruta);
});
```

⍰ Métodos Funcionales (map, filter, reduce, etc.)

map()

Devuelve un nuevo array aplicando una función a cada elemento.

```
let numeros = [1, 2, 3];
let dobles = numeros.map(n => n * 2); // [2, 4, 6]
```

---

filter()

Devuelve un nuevo array con los elementos que cumplan una condición.

```
let mayores = numeros.filter(n => n > 1); // [2, 3]
```

some()

Devuelve true si al menos un elemento cumple la condición.

```
numeros.some(n => n > 2); // true
```

every()

Devuelve true si todos los elementos cumplen la condición.

```
numeros.every(n => n > 0); // true
```

reduce()

Reduce el array a un solo valor acumulado.

```
let suma = numeros.reduce((acc, n) => acc + n, 0); // 6
```

⍰ Métodos Modernos (ES2022+)

at()

Accede al valor en una posición, permite índices negativos.

```
let letras = ['a', 'b', 'c'];
letras.at(-1); // 'c'
```

toSorted()

Igual que sort(), pero no modifica el array original.

```
let copiaOrdenada = numeros.toSorted();
```

toReversed()

Igual que reverse(), pero no modifica el array original.

```
let copiaReversa = numeros.toReversed();
```